Page 1 - Local AI Assistant Overview

Project Aurora is a local AI assistant designed to run primarily on a personal computer without relying on cloud LLM APIs.

The assistant uses Ollama as its initial inference backend. Its development model is Qwen3.5 2B, while larger models can be selected when more reasoning quality is needed.

The system is intentionally modular. The Assistant class manages conversation state, while an LLM provider interface separates application logic from the inference engine.

A future version may communicate with an ESP32 desktop device over Wi-Fi. The ESP32 will handle sensors, audio, camera input, and physical interaction, while the main computer performs LLM inference.

Page 2 - Basic RAG Architecture

Retrieval-Augmented Generation, or RAG, allows the assistant to answer questions using information stored in external documents.

The basic pipeline contains five main steps: document loading, text chunking, embedding generation, vector storage, and semantic retrieval.

When a user asks a question, the query is converted into an embedding. The vector database searches for chunks whose embeddings are close to the query embedding.

The retrieved chunks are inserted into the LLM prompt as context. This helps ground the answer in the user's documents instead of depending only on the model's pretrained knowledge.

Every chunk should preserve metadata such as source filename and page number so the assistant can later provide citations.

Page 3 - Future Retrieval Improvements

The first RAG implementation should remain simple so that retrieval problems are easy to diagnose.

A later version can add metadata filtering, hybrid retrieval, and reranking. Hybrid retrieval combines semantic vector search with lexical or sparse retrieval.

A reranker receives an initial set of candidate chunks and scores them again with a more accurate relevance model. The final prompt should contain only the most relevant chunks.

Long-term memory should remain separate from document retrieval. Structured memories can be stored in a database and retrieved when relevant, while Obsidian can optionally provide a human-readable Markdown view.

RAG quality should eventually be evaluated with a small test set containing questions, expected source passages, and expected answers.